Programming Fundamentals Using C++

Huma Israr

Department Of Information Engineering Technology

National Skills University, Islamabad.

huma.israr@nsu.edu.pk

Fall 2024, BS-CS

Week 05

Lecture – 01

Lecture – 02

Operators in C++

Semester Calendar

Class Schedule

Lab Hours (Practice)

Tuesday (09:00 to 10:30)
Wednesday (09:30 to 12:30)
Wednesday (03:00 to 04:30)

C++ Operators

- Operators are symbols that perform operations on variables and values.
- For Example , + operator to add together two values:

```
int x = 100 + 50;
```

- Although the + operator is often used to add together two values, like in the example above, it can also be used to add together a variable and a value, or a variable and another variable.

```
int sum1 = 100 + 50;        // 150 (100 + 50)
int sum2 = sum1 + 250;      // 400 (150 + 250)
int sum3 = sum2 + sum2;     // 800 (400 + 400)
```

C++ Operators:

- Operators in C++ can be classified into 6 types:
- Arithmetic Operators
- Assignment Operators
- Relational Operators
- Logical Operators
- Bitwise Operators
- Other Operators

Assignment operator (=)

- The assignment operator assigns a value to a variable.

```
x = 5;
```

- This statement assigns the integer value 5 to the variable x. The assignment operation always takes place from right to left, and never the other way around:

```
x = y;
```

- This statement assigns to variable x the value contained in variable y. The value of x at the moment this statement is executed is lost and replaced by the value of y.

Assignment Operators

- Assignment operations are expressions that can be evaluated.

```
y = 2 + (x = 5);
```

- Assignment Operator could be used as

```
x = 5;
y = 2 + x;
```

- The following expression is also valid in C++:

```
x = y = z = 5;
```

- It assigns 5 to the all three variables: x, y and z; always from right-to-left.

Arithmetic Operators:

- These are the operators used to perform arithmetic/mathematical operations on operands.

Operator A + B → Operands

- Arithmetic operators are of two types:
- Unary Operators: Operators that operate or work with a single operand are unary operators. For example: (++ , --)

a--

- Binary Operators: Operators that operate or work with two operands are binary operators. For example: (+ , - , * , /)

Unary Operator

```
#include<iostream>
using namespace std;
int main()
{
int a;  // Variable declaration
a = 5;    // Actual initialization
a=a+1;
cout<<"a = "<<a;
return 0;
}
```

Unary Operator

```
#include<iostream>
using namespace std;
int main()
{
int a; // Variable declaration
a = 5;    // Actual initialization
a++;
cout<<"a = "<<a;
return 0;
}
```

Arithmetic operators (+, -, *, /, %)

- Five arithmetical operations are supported by C++
- Arithmetic operator (+,-,*,/,%) directs the computer to perform certain mathematical manipulations .
- +, -, *, and / can be used with integral and floating-point data types.
- The expression x = x+3; equivalent to x+=3;
- Similarly

x = x-5;

x -=5;

Operators

+	Addition	A+B		
-	Subtraction	A-B		
*	Multiplication	A*B		
/	Division	A/B		
%	Modulus Division	A%B		
+=	Add and Assign	A+=B	→	A=A+B
-=	Subtract and Assign	A-=B	→	A=A-B
*=	Multiply and Assign	A*=B	→	A=A*B
/=	Divided and Assign	A/=B	→	A=A/B
%=	Mod and Assign	A%=B	→	A=A%B

Increment(++) & Decrement(– –) Operators :

- C++ supports 2 useful operators namely

  1. **Increment ++ operators**
  2. **Decrement – – operators**

- ++ Increment operator: increment variable by 1
- --Decrement operator: decrement variable by 1
- Pre-increment: ++variable
- Post-increment: variable++
- Pre-decrement: --variable
- Post-decrement: variable--

Increment Operators	Decrement Operators
Increment Operator adds 1 to the operand.	Decrement Operator subtracts 1 from the operand.
Postfix increment operator means the expression is evaluated first using the original value of the variable and then the variable is incremented (increased).	Postfix decrement operator means the expression is evaluated first using the original value of the variable and then the variable is decremented (decreased).
Prefix increment operator means the variable is incremented first and then the expression is evaluated using the new value of the variable.	Prefix decrement operator means the variable is decremented first and then the expression is evaluated using the new value of the variable.
Generally, we use this in decision making and looping.	This is also used in decision making and looping.

Pre Increment Operator

```
#include<iostream>
int main()
{
int x,i;
i=10;
x=++i;
cout<<"x: "<<x;
cout<<"i: "<<i;
return 0;
}
```

Post-Increment Operator

```
#include<iostream>
int main()
{
int x,i;
i=10;
x=i++;
cout<<"x: "<<x;
cout<<"i: "<<i;
return 0;
}
```

Relational Operators:

- These are used for the comparison of the values of two operands. For example, checking if one operand is equal to the other operand or not, an operand is greater than the other operand or not, etc. Some of the relational operators are (>, < , ==, >= , <= , !=).

```
// checks if a is greater than b
a > b;
```

> and < Operators

- Greater than operator: Represented as ‘>’, the greater than operator checks whether the first operand is greater than the second operand or not. If so, it returns true. Otherwise it returns false. For example, 6>5 will return true.

A > b ;

- Less than operator: Represented as ‘<‘, the less than operator checks whether the first operand is lesser than the second operand. If so, it returns true. Otherwise it returns false. For example, 6<5 will return false.

A < b ;

>= and <= Operators

- Greater than or equal to operator: Represented as '>=', the greater than or equal to operator checks whether the first operand is greater than or equal to the second operand. If so, it returns true else it returns false. For example, 5>=5 will return true.

A >= B

- Less than or equal to operator: Represented as '<=', the less than or equal to Operator checks whether the first operand is less than or equal to the second operand. If so, it returns true else false. For example, 5<=5 will also return true

A <= B

== and != Operators

- Equal to operator: Represented as '==', the equal to operator checks whether the two given operands are equal or not. If so, it returns true. Otherwise it returns false. For example, 5==5 will return true.

  x == y

- Not equal to operator: Represented as '!=', the not equal to operator checks whether the two given operands are equal or not. If not, it returns true. Otherwise it returns false. It is the exact boolean complement of the '==' operator. For example, 5!=5 will return false.

  X != Y

```
int x = 10;
int y = 15;
int z = 10;
x == y   // false
x == z   // true
```

```
int x = 10;
int y = 15;
int z = 10;
x != y   // true
x != z   // false
```

```
int x = 10;
int y = 15;
x > y   // false
y > x   // true
```

```
int x = 10;
int y = 15;
x < y   // true
y < x   // false
```

```
int x = 10;
int y = 15;
int z = 10;

x >= y   // false
y >= x   // true
z >= x   // true
```

```
int x = 10;
int y = 15;
int z = 10;

x <= y   // true
y <= x   // false
z <= x   // false
```

Logical Operators:

- Logical Operators are used to combine two or more conditions/constraints or to complement the evaluation of the original condition in consideration.
- Logical AND && (Expression 1 and Expression 2)
- Logical OR || (Expression 1 OR Expression 2)
- Logical Not ! (Not Expression)

Logical Operators:

- Logical Operators are used to combine two or more conditions/constraints or to complement the evaluation of the original condition in consideration.
- The result of the operation of a logical operator is a boolean value either true or false. For example, the logical AND represented as '&&' operator in C or C++ returns true when both the conditions under consideration are satisfied. Otherwise, it returns false. Therefore, a && b returns true when both a and b are true (i.e. non-zero).

Bitwise Operators:

- The Bitwise operators are used to perform bit-level operations on the operands. The operators are first converted to bit-level and then the calculation is performed on the operands. The mathematical operations such as addition, subtraction, multiplication, etc. can be performed at bit-level for faster processing. For example, the bitwise AND represented as & operator in C or C++ takes two numbers as operands and does AND on every bit of two numbers. The result of AND is 1 only if both bits are 1.

Bitwise Operators:

6 bitwise operators included in C++.

Operator	Description
&	Bitwise AND Operator
|	Bitwise OR Operator
^	Bitwise XOR Operator
~	Bitwise Complement Operator
<<	Bitwise Shift Left Operator
>>	Bitwise Shift Right Operator.

Bitwise Operators:

- Bit wise AND

a	b	a & b
0	0	0
0	1	0
1	0	0
1	1	1

- Bitwise OR

a	b	a | b
0	0	0
0	1	1
1	0	1
1	1	1

Bit Wise AND

12 = 00001100 (In Binary)
25 = 00011001 (In Binary)
Bitwise AND Operation of 12 and 25

```
  00001100
& 00011001
  ________
  00001000  = 8 (In decimal)
```

Bit Wise OR

12 = 00001100 (In Binary)
25 = 00011001 (In Binary)
Bitwise OR Operation of 12 and 25

```
  00001100
| 00011001
  ________
  00011101  = 29 (In decimal)
```

Bitwise AND

```
#include <iostream>
using namespace std;
int main() {
    // declare variables
    int a = 12, b = 25;

    cout << "a = " << a << endl;
    cout << "b = " << b << endl;
    cout << "a & b = " << (a & b) << endl;

    return 0;
}
```

Any Question